Practical : 7	Implementation of Making a Change / Coin Change Problem using Dynamic Programming.
Date : 12/08/2026	

Objective :- To implement the Coin Change Problem in C++ using Dynamic Programming to compute both the minimum number of coins needed to make a given amount and the number of distinct ways to make that amount, and to analyse the time and space complexity.

Program :-

```
#include <iostream>
#include <vector>
#include <climits>
using namespace std;

// ---------------- Coin Change - Minimum Coins (DP, O(n * amount)) ----------------
int minCoins(vector<int>& coins, int amount) {
vector<int> dp(amount + 1, INT_MAX);
dp[0] = 0;
for (int a = 1; a <= amount; a++) {
for (int c : coins) {
if (c <= a && dp[a - c] != INT_MAX)
dp[a] = min(dp[a], dp[a - c] + 1);
}
}
return dp[amount] == INT_MAX ? -1 : dp[amount];
}

// ---------------- Coin Change - Number of Ways (DP, O(n * amount)) ----------------
long long countWays(vector<int>& coins, int amount) {
vector<long long> dp(amount + 1, 0);
dp[0] = 1;
for (int c : coins) {
for (int a = c; a <= amount; a++)
dp[a] += dp[a - c];
}
return dp[amount];
}

int main() {
vector<int> coins = {1, 5, 10, 25};
int amount = 63;

int fewest = minCoins(coins, amount);
long long ways = countWays(coins, amount);

cout << "Amount to make: " << amount << endl;
cout << "Minimum coins needed: " << fewest << endl;
cout << "Number of distinct ways to make amount: " << ways << endl;
```

```
return 0;
}
```

Output :-

```
Amount to make: 63
Minimum coins needed: 6
Number of distinct ways to make amount: 73
```

Analysis:-

Time & Space Complexity (n = number of coin denominations, amount = target value):

Method	Time Complexity	Space Complexity
Min Coins - Dynamic Programming	O(n × amount)	O(amount)
Count Ways - Dynamic Programming	O(n × amount)	O(amount)

The Coin Change problem has two common variants, both solved here with bottom-up DP over the target amount. For the minimum-coins variant, dp[a] holds the fewest coins needed to make amount a, computed as 1 plus the best of dp[a-c] over every coin denomination c that fits, so filling all 'amount' entries against n coins costs O(n × amount) time and O(amount) space. For the count-the-ways variant, dp[a] instead accumulates the number of distinct combinations that sum to a, but the coin loop is placed on the outside so each coin is considered once per amount — this avoids counting the same combination in a different order as a separate way, which is essential for a correct combination count rather than a permutation count; this variant has the same O(n × amount) time and O(amount) space bounds. For the sample instance with coins {1, 5, 10, 25} and amount 63, the DP correctly finds that the fewest coins needed is 6 (e.g. 25+25+10+1+1+1) and that there are 73 distinct combinations of these coins that sum to 63.

Conclusion :- The Coin Change Problem was successfully implemented in C++ covering both the minimum-coins and count-the-ways variants using bottom-up Dynamic Programming over the target amount. The program correctly computed a minimum of 6 coins and 73 distinct ways to make an amount of 63 from denominations {1, 5, 10, 25}, confirming the expected O(n × amount) time and O(amount) space behaviour for both variants. Ordering the coin and amount loops correctly was shown to matter: iterating coins on the outer loop for the ways-counting variant is what guarantees combinations rather than permutations are counted.